```
def calculate_cost(state):
    """Count pairs of queens attacking each other."""
    n = len(state)
    conflicts = 0
    for i in range(n):
        for j in range(i + 1, n):
            # Same row or same diagonal
            if (state[i] == state[j] or
                abs(state[i] - state[j]) == abs(i - j)):
                conflicts += 1
    return conflicts

def get_best_neighbor(state):
    """Find the neighbor with the lowest cost."""
    n = len(state)
    best_state = state[:]
    best_cost = calculate_cost(state)
    for col in range(n):
        original_row = state[col]
        for row in range(1, n + 1):
            if row == original_row:
                continue
            neighbor = state[:]
            neighbor[col] = row
            cost = calculate_cost(neighbor)
            if cost < best_cost:
                best_state = neighbor
                best_cost = cost
    return best_state, best_cost

def hill_climbing(initial_state):
    current = initial_state[:]
    print("Initial state:", current)
    print("Initial cost:", calculate_cost(current))
    while True:
        neighbor, neighbor_cost = get_best_neighbor(current)
        # No improvement is possible
```

```
        if neighbor_cost >= calculate_cost(current):
            break
        current = neighbor
        print("Next state:", current)
        print("Cost:", neighbor_cost)
        if neighbor_cost == 0:
            print("Goal state reached!")
            return current
    if calculate_cost(current) == 0:
        print("Goal state reached!")
    else:
        print("Local minimum reached; no better neighbor.")
    return current

# Initial configuration from the example
initial_state = [1, 1, 1, 1]
solution = hill_climbing(initial_state)
print("Final configuration:", solution)
print("Final cost:", calculate_cost(solution))
```

OUTPUT:

```
================================================
Initial state: [1, 1, 1, 1]
Initial cost: 6
Next state: [1, 4, 1, 1]
Cost: 3
Next state: [2, 4, 1, 1]
Cost: 1
Next state: [2, 4, 1, 3]
Cost: 0
Goal state reached!
Final configuration: [2, 4, 1, 3]
Final cost: 0
>>
```